Operating Systems Week 4 Lecture 4

Junaid Abdul Wahid

Previous

- Role of Process Control Block
- Switching Processes

Today Agenda

- Concurrency: Mutual Exclusion and Synchronization

Roadmap



- Principals of Concurrency
- Mutual Exclusion: Hardware Support
- Semaphores
- Monitors
- Message Passing
- Readers/Writers Problem

Multiple Processes

Central to the design of modern Operating Systems is managing multiple processes

- Multiprogramming
- Multiprocessing
- Distributed Processing

Big Issue is Concurrency

- Managing the interaction of all of these processes

Concurrency

Concurrency arises in:

- Multiple applications
  - Sharing time
- Structured applications
  - Extension of modular design
- Operating system structure
  - OS themselves implemented as a set of processes or threads

Key Terms

Table 5.1 Some Key Terms Related to Concurrency

atomic operation	A sequence of one or more statements that appears to be indivisible; that is, no other process can see an intermediate state or interrupt the operation.
critical section	A section of code within a process that requires access to shared resources and that must not be executed while another process is in a corresponding section of code.
deadlock	A situation in which two or more processes are unable to proceed because each is waiting for one of the others to do something.
livelock	A situation in which two or more processes continuously change their states in response to changes in the other process(es) without doing any useful work.
mutual exclusion	The requirement that when one process is in a critical section that accesses shared resources, no other process may be in a critical section that accesses any of those shared resources.
race condition	A situation in which multiple threads or processes read and write a shared data item and the final result depends on the relative timing of their execution.
starvation	A situation in which a runnable process is overlooked indefinitely by the scheduler; although it is able to proceed, it is never chosen.

Interleaving and Overlapping Processes

Earlier (Ch2) we saw that processes may be interleaved on uniprocessors

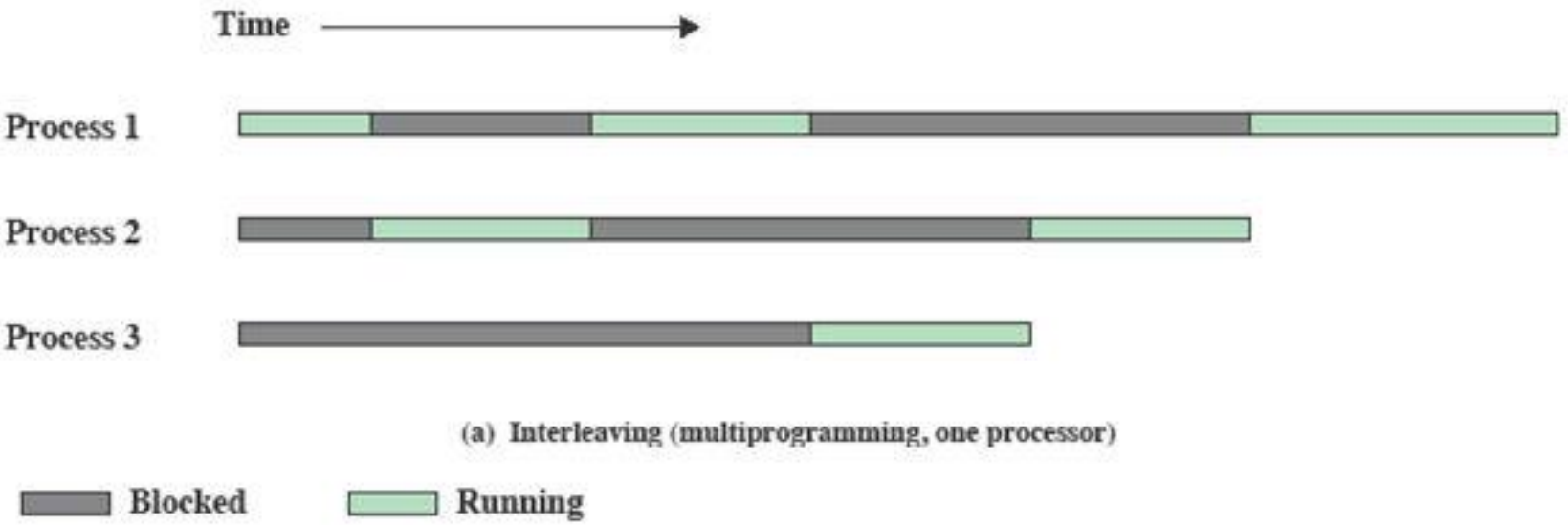
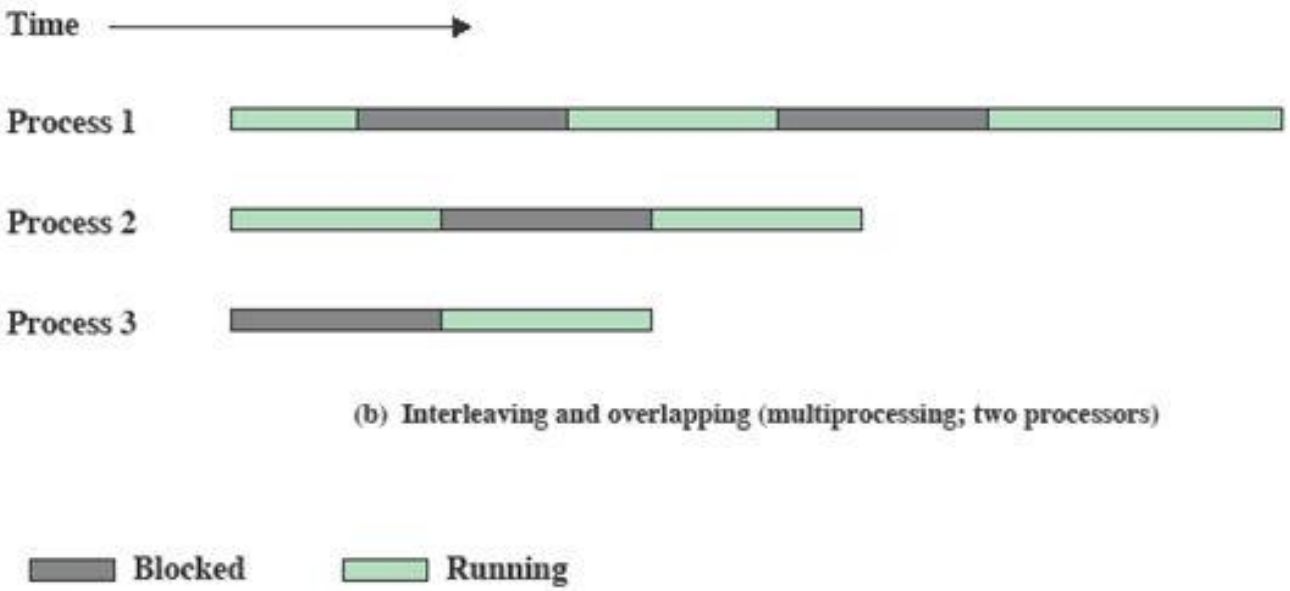


Figure 2.12 Multiprogramming and Multiprocessing

Interleaving and Overlapping Processes

And not only interleaved but overlapped on multi-processors



(b) Interleaving and overlapping (multiprocessing; two processors)

Figure 2.12 Multiprogramming and Multiprocessing

Difficulties of Concurrency

Sharing of global resources
Optimally managing the allocation of resources
Difficult to locate programming errors as results are not deterministic and reproducible.

A Simple Example

```
void echo()
{
        chin = getchar();
        chout = chin;
        putchar(chout);
}
```

Enforce Single Access

If we enforce a rule that only one process may enter the function at a time then:

P1 & P2 run on separate processors

P1 enters echo first,

- P2 tries to enter but is blocked – P2 suspends

P1 completes execution

- P2 resumes and executes echo

Race Condition

A race condition occurs when

- Multiple processes or threads read and write data items
- They do so in a way where the final result depends on the order of execution of the processes.

The output depends on who finishes the race last.

Operating System Concerns

What design and management issues are raised by the existence of concurrency?

The OS must

- Keep track of various processes
- Allocate and de-allocate resources
- Protect the data and resources against interference by other processes.
- Ensure that the processes and outputs are independent of the processing speed

Process Interaction

Table 5.2 Process Interaction

Degree of Awareness	Relationship	Influence That One Process Has on the Other	Potential Control Problems
Processes unaware of each other	Competition	• Results of one process independent of the action of others • Timing of process may be affected	• Mutual exclusion • Deadlock (renewable resource) • Starvation
Processes indirectly aware of each other (e.g., shared object)	Cooperation by sharing	• Results of one process may depend on information obtained from others • Timing of process may be affected	• Mutual exclusion • Deadlock (renewable resource) • Starvation • Data coherence
Processes directly aware of each other (have communication primitives available to them)	Cooperation by communication	• Results of one process may depend on information obtained from others • Timing of process may be affected	• Deadlock (consumable resource) • Starvation

Competition among Processes for Resources

Three main control problems:
Need for Mutual Exclusion
- Critical sections

Deadlock
Starvation

Requirements for Mutual Exclusion

Only one process at a time is allowed in the critical section for a resource
A process that halts in its noncritical section must do so without interfering with other processes
No deadlock or starvation

Requirements for Mutual Exclusion

A process must not be delayed access to a critical section when there is no other process using it
No assumptions are made about relative process speeds or the number of processes
A process remains inside its critical section for a finite time only

Roadmap



- Principals of Concurrency
- Mutual Exclusion: Hardware Support
- Semaphores
- Monitors
- Message Passing
- Readers/Writers Problem

Disabling Interrupts

Uniprocessors only allow interleaving

Interrupt Disabling

- A process runs until it invokes an operating system service or until it is interrupted
- Disabling interrupts guarantees mutual exclusion
- Will not work in multiprocessor architecture

Pseudo-Code

```
while (true) {
    /* disable interrupts */;
    /* critical section */;
    /* enable interrupts */;
    /* remainder */;
}
```

Special Machine Instructions

Compare&Swap Instruction

also called a “compare and exchange instruction”

Exchange Instruction

Compare&Swap Instruction

```
int compare_and_swap (int *word,
     int testval, int newval)
{
   int oldval;
   oldval = *word;
   if (oldval == testval) *word = newval;
   return oldval;
}
```

Mutual Exclusion (fig 5.2)

```
/* program mutualexclusion */
const int n = /* number of processes */;
int bolt;
void P(int i)
{
   while (true) {
     while (compare_and_swap(bolt, 0, 1) == 1)
          /* do nothing */;
      /* critical section */;
      bolt = 0;
      /* remainder */;
   }
}
void main()
{
   bolt = 0;
   parbegin (P(1), P(2), ... ,P(n));

}
```

(a) Compare and swap instruction

Exchange instruction

```
void exchange (int register, int
memory)
{
   int temp;
   temp = memory;
   memory = register;
   register = temp;
}
```

Exchange Instruction (fig 5.2)

```
   /* program mutualexclusion */
int const n = /* number of processes**/;
int bolt;
void P(int i)
{
   int keyi = 1;
   while (true) {
      do exchange (keyi, bolt)
      while (keyi != 0);
      /* critical section */;
      bolt = 0;
      /* remainder */;
   }
}
void main()
{
   bolt = 0;
   parbegin (P(1), P(2), ..., P(n));
}
```

(b) Exchange instruction

Hardware Mutual Exclusion: Advantages

Applicable to any number of processes on either a single processor or multiple processors sharing main memory
It is simple and therefore easy to verify
It can be used to support multiple critical sections

Hardware Mutual Exclusion: Disadvantages

Busy-waiting consumes processor time

Starvation is possible when a process leaves a critical section and more than one process is waiting.

- Some process could indefinitely be denied access.

Deadlock is possible

Semaphore

Semaphore:

- An integer value used for signalling among processes.

Only three operations may be performed on a semaphore, all of which are atomic:

- initialize,
- Decrement (`semWait`)
- increment. (`semSignal`)

Semaphore Primitives

```
struct semaphore {
      int count;
      queueType queue;
};
void semWait(semaphore s)
{
      s.count--;
      if (s.count < 0) {
            /* place this process in s.queue */;
            /* block this process */;
      }
}
void semSignal(semaphore s)
{
      s.count++;
      if (s.count <= 0) {
            /* remove a process P from s.queue */;
            /* place process P on ready list */;
      }
}
```

Figure 5.3 A Definition of Semaphore Primitives

Binary Semaphore Primitives

```
struct binary_semaphore {
      enum {zero, one} value;
      queueType queue;
};
void semWaitB(binary_semaphore s)
{
      if (s.value == one)
            s.value = zero;
      else {
               /* place this process in s.queue */;
               /* block this process */;
      }
}
void semSignalB(semaphore s)
{
      if (s.queue is empty())
            s.value = one;
      else {
               /* remove a process P from s.queue */;
               /* place process P on ready list */;
      }
}
```

Figure 5.4 A Definition of Binary Semaphore Primitives

Strong/Weak Semaphore

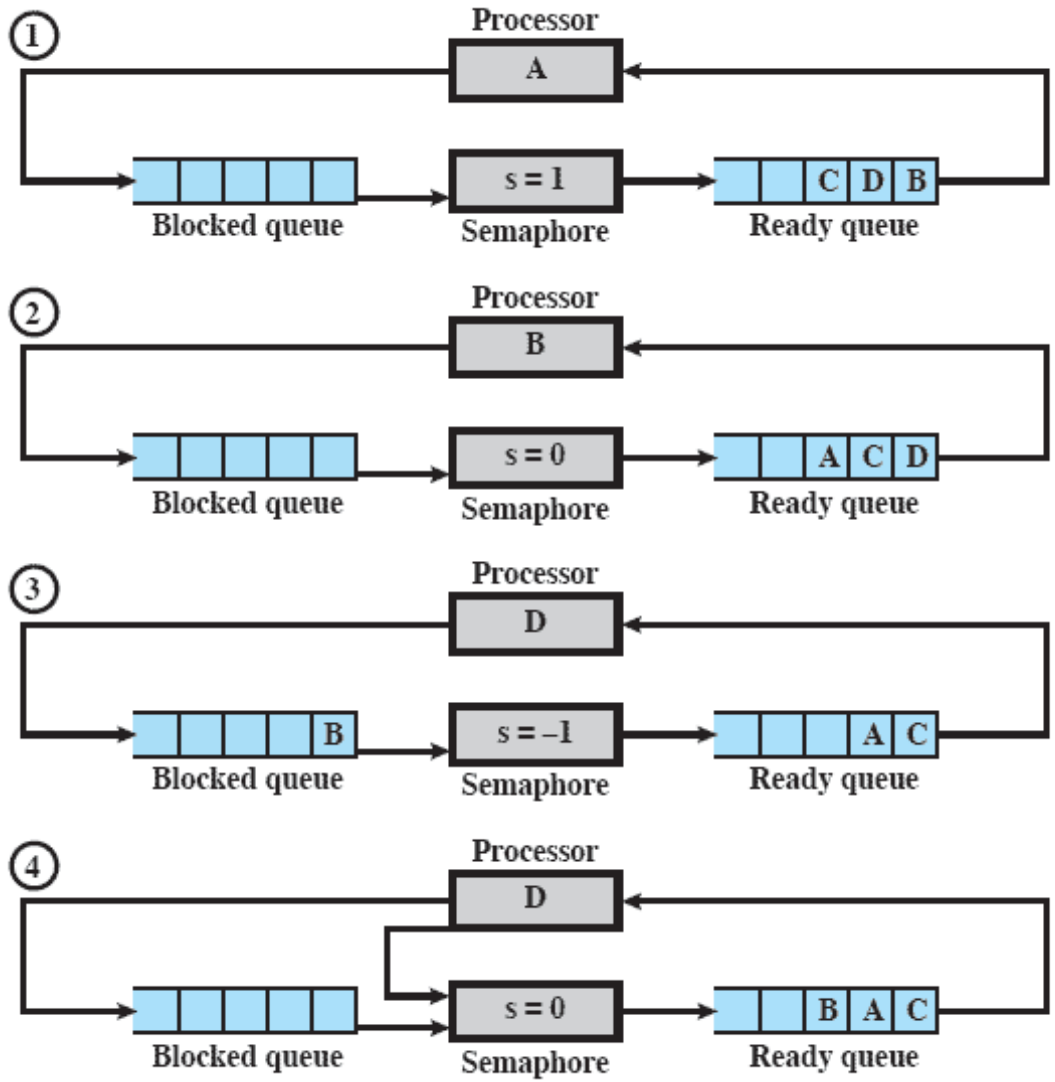
A queue is used to hold processes waiting on the semaphore

In what order are processes removed from the queue?

Strong Semaphores use FIFO

Weak Semaphores don't specify the order of removal from the queue

Example of Strong Semaphore Mechanism



Example of Semaphore Mechanism

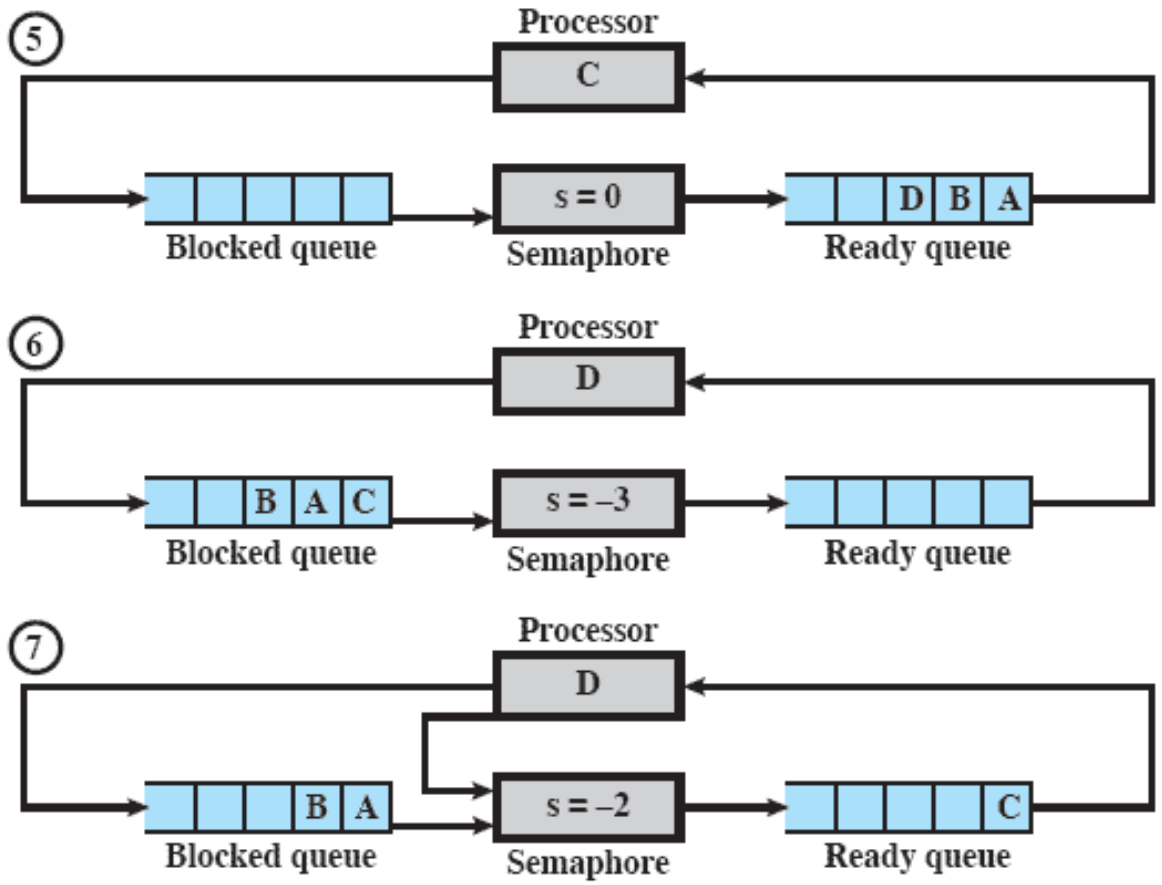


Figure 5.5 Example of Semaphore Mechanism

Mutual Exclusion Using Semaphores

```
/* program mutualexclusion */
const int n = /* number of processes  */;
semaphore s = 1;
void P(int i)
{
     while (true) {
          semWait(s);
          /* critical section   */;
          semSignal(s);
          /* remainder   */;
     }
}
void main()
{
     parbegin (P(1), P(2), . . ., P(n));
}
```

Figure 5.6 Mutual Exclusion Using Semaphores

Processes Using Semaphore

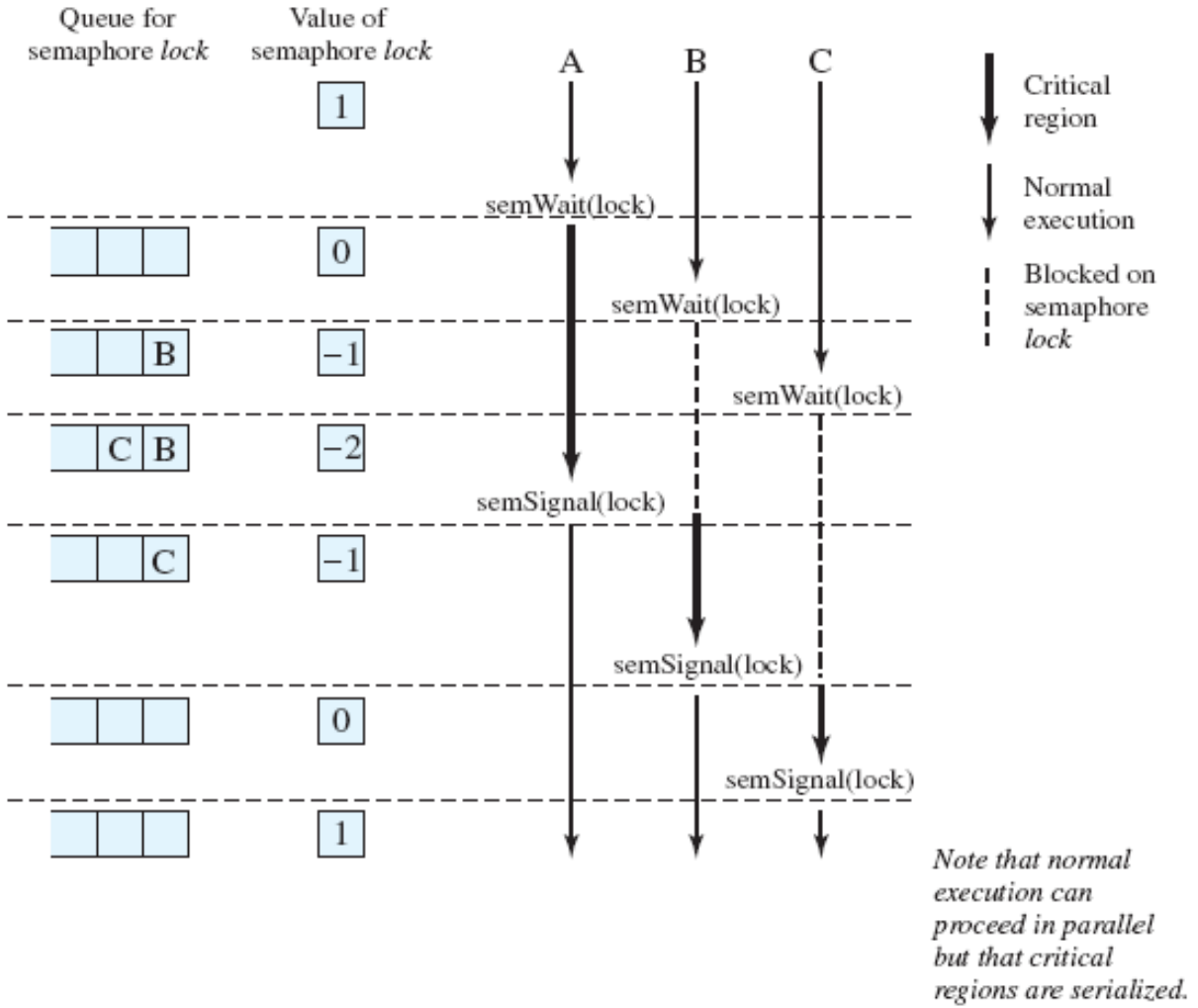


Figure 5.7 **Processes Accessing Shared Data Protected by a Semaphore**

Producer/Consumer Problem

General Situation:

One or more producers are generating data and placing these in a buffer

A single consumer is taking items out of the buffer one at time

Only one producer or consumer may access the buffer at any one time

The Problem:

Ensure that the Producer can't add data into full buffer and consumer can't remove data from empty buffer